PodpisECC

Instrukcja użytkowania, weryfikacji wyniku
i dalszego rozwoju

System podpisu cyfrowego ECC
ECDSA / secp256r1 / SHA-256 / OpenSSL

2026

Spis treści

1 Czy coś jeszcze trzeba? 2

2 Uruchomienie aplikacji 2

3 Jak używać — krok po kroku 3
3.1 Scenariusz A — podpis i weryfikacja poprawna (happy-path) . . . 3
3.2 Scenariusz B — wykrycie zmiany dokumentu . . . 3
3.3 Scenariusz C — niewłaściwy klucz . . . 3
3.4 Scenariusz D — błędy użytkownika (powiadomienia) . . . 4
3.5 Scenariusz E — pomiar wydajności . . . 4

4 Jak weryfikować wynik (checklista na obronę) 4

5 Automatyczne testy 4

6 Co jeszcze można poprawić / rozwinąć 5
6.1 Szybkie ulepszenia (1–2 wieczory) . . . 5
6.2 Rozszerzenia funkcjonalne . . . 5
6.3 Bezpieczeństwo (poziom zaawansowany) . . . 5
6.4 Czego nie trzeba robić na zaliczenie . . . 5

7 Struktura projektu (skrót) 6

8 Szybka ściąga 6

1 Czy coś jeszcze trzeba?

Na zaliczenie projektu **masz już komplet**:

- działającą aplikację webową (C++ + OpenSSL),
- generowanie kluczy, podpis, weryfikację, SHA-256,
- hasło do klucza prywatnego, limity uploadu, powiadomienia,
- pomiar ECC vs RSA-3072,
- skrypt testów automatycznych,
- dokumentację LaTeX/PDF w folderze `korostil`.

Opcjonalnie (niekonieczne, ale ładnie wygląda na obronie):

- 2–3 zrzuty ekranu UI w dokumentacji projektowej,
- krótka prezentacja / demo na żywo (happy-path + test negatywny),
- uzupełnienie wniosków po pomiarach z benchmarku.

Nie musisz dodawać PKI, HTTPS ani GUI desktopowego — to było poza zakresem.

2 Uruchomienie aplikacji

1. Otwórz PowerShell.
2. Przejdź do katalogu projektu:

```
cd "C:\Users\Adrian\Desktop\Podpis cyfrowy ecc"
```

3. Jeśli jeszcze nie budowałeś albo zmieniłeś kod:

```
cmake -S . -B build -G "MinGW Makefiles" ^
  -DCMAKE_BUILD_TYPE=Release ^
  -DOPENSSL_ROOT_DIR="C:/Strawberry/c"
cmake --build build
```

4. Uruchom serwer:

```
.\build\bin\podpis_ecc.exe
```

5. W przeglądarce otwórz: http://127.0.0.1:8080/
6. Zatrzymanie: w oknie serwera `Ctrl+C` albo zamknięcie procesu.

Inny port (np. gdy 8080 zajęty):

```
.\build\bin\podpis_ecc.exe 9090
```

wtedy UI: http://127.0.0.1:9090/

3 Jak używać — krok po kroku

3.1 Scenariusz A — podpis i weryfikacja poprawna (happy-path)

1. W sekcji **1. Generuj klucze** wpisz hasło (zalecane, min. 6 znaków) dwa razy.
2. Kliknij **Generuj parę kluczy**.
3. Pobierz `private.pem` i `public.pem` (linki w wyniku).
4. Przygotuj dowolny plik tekstowy, np. `umowa.txt`.
5. W sekcji **2. Podpisz dokument**:
   - upuść / wybierz dokument,
   - upuść / wybierz `private.pem`,
   - wpisz hasło (jeśli klucz zaszyfrowany),
   - opcjonalnie: **Podgląd SHA-256**,
   - kliknij **Podpisz i pobierz .sig**.
6. Zapisz pobrany `document.sig`.
7. W sekcji **3. Zweryfikuj podpis** wgraj:
   - ten sam dokument,
   - `public.pem`,
   - `document.sig`.
8. Kliknij **Weryfikuj**.

Oczekiwany wynik: toast i panel — *podpis PRAWIDŁOWY* (kolor zielony).

3.2 Scenariusz B — wykrycie zmiany dokumentu

1. Po udanym podpisie zmień 1 znak w pliku dokumentu (lub wgraj inną wersję).
2. Zweryfikuj **starym** podpisem i tym samym kluczem publicznym.

Oczekiwany wynik: *podpis NIEPRAWIDŁOWY* — to dowód kontroli integralności.

3.3 Scenariusz C — niewłaściwy klucz

1. Wygeneruj **drugą** parę kluczy.
2. Dokument podpisany kluczem A zweryfikuj kluczem publicznym B.

Oczekiwany wynik: *podpis NIEPRAWIDŁOWY*.

3.4 Scenariusz D — błędy użytkownika (powiadomienia)

Sprawdź, że aplikacja komunikuje konkretną przyczynę:

- brak pliku dokumentu / klucza / podpisu,
- klucz publiczny wgrany jako prywatny (i odwrotnie),
- złe hasło do zaszyfrowanego PEM,
- zaszyfrowany klucz bez podanego hasła,
- złe rozszerzenie pliku (`.pem`, `.sig`, ...).

3.5 Scenariusz E — pomiar wydajności

W sekcji **Wydajność: ECC vs RSA-3072** ustaw 2–3 rundy i kliknij **Uruchom pomiar**. Porównaj czasy keygen / sign / verify oraz rozmiar podpisu (ECC zwykle krótszy i szybszy w keygen).

4 Jak weryfikować wynik (checklista na obronę)

Test	Oczekiwanie
Generowanie kluczy	dwa pliki PEM, opcjonalnie ENCRYPTED
Podpis	plik `.sig`, niepusty
Weryfikacja OK	`valid: true` / PRAWIDŁOWY
Zmiana dokumentu	NIEPRAWIDŁOWY
Zły klucz publiczny	NIEPRAWIDŁOWY
Złe hasło	komunikat błędu (HTTP 400)
Historia sesji	zdarzenia bez treści klucza prywatnego

To wystarczy, żeby pokazać: **uwierzytelnienie**, **integralność**, **niezaprzeczalność** (przy tajności klucza prywatnego).

5 Automatyczne testy

Serwer musi być uruchomiony (`podpis_ecc.exe` na porcie 8080).

W **drugim** oknie PowerShell:

```
cd "C:\Users\Adrian\Desktop\Podpis cyfrowy ecc"
powershell -ExecutionPolicy Bypass -File .\scripts\test_flow.ps1
```

Skrypt sprawdza m.in.:

- dostępność UI,
- generowanie kluczy z hasłem,
- SHA-256,
- podpis + weryfikację pozytywną,

- weryfikację negatywną po zmianie dokumentu,
- odrzucenie złego hasła,
- endpoint benchmarku.

Sukces: komunikat `Wszystkie testy przeszły.` oraz linie `OK ...`.

6 Co jeszcze można poprawić / rozwinąć

6.1 Szybkie ulepszenia (1–2 wieczory)

- zrzuty ekranu do dokumentacji LaTeX,
- tabela wyników benchmarku wpisana do rozdziału wniosków,
- eksport historii sesji do pliku JSON/CSV (bez kluczy),
- ciemniejszy / jaśniejszy motyw UI.

6.2 Rozszerzenia funkcjonalne

- podpis wielu plików naraz (batch),
- weryfikacja "przeciągnij folder",
- zapis metadanymi: kto / kiedy (lokalny log, nie PKI),
- porównanie z ECDSA na `secp256k1` (Bitcoin) obok P-256,
- prosty tryb "tylko weryfikacja" dla drugiej osoby (bez generowania).

6.3 Bezpieczeństwo (poziom zaawansowany)

- magazyn klucza w systemie (DPAPI / Credential Manager),
- HTTPS lokalnie (opcjonalnie),
- HSM / token USB — poza typowym zakresem przedmiotu.

6.4 Czego nie trzeba robić na zaliczenie

- pełne PKI / certyfikaty CA,
- kwalifikowany podpis elektroniczny,
- własna implementacja arytmetyki krzywej zamiast OpenSSL.

7 Struktura projektu (skrót)

```
Podpis cyfrowy ecc/
  src/           C++ (OpenSSL + serwer HTTP)
  web/           UI (HTML/CSS/JS)
  scripts/       test_flow.ps1
  docs/          ta instrukcja (PDF)
  build/bin/     podpis_ecc.exe
```

Dokumentacja akademicka: folder `korostil` na Pulpicie (`build/dokument.pdf`).

8 Szybka ściąga

1. `.\build\bin\podpis_ecc.exe`
2. Przeglądarka → http://127.0.0.1:8080/
3. Generuj klucze → podpisz → weryfikuj OK
4. Zmień dokument → weryfikuj FAIL
5. `.\scripts\test_flow.ps1` przy działającym serwerze

Koniec instrukcji. Powodzenia na zaliczeniu.